Electronic System Level Design

Smart Room Controller using SystemC TLM-2.0

Technical Report

Name : Neethu Jaisan
Github: https://github.com/Neethu-Jaisan/Smart-Room-Controller-System-C-TLM-Modeling

1. Introduction

1.1 Motivation

Modern embedded systems grow in complexity rapidly, making it increasingly costly to discover design flaws late in the development cycle. Validating system behavior early — before RTL or hardware implementation — reduces re-spin risk and development time. Transaction-Level Modeling (TLM) within the SystemC framework provides an ideal abstraction for this purpose, enabling fast simulation and architectural exploration.

This project demonstrates a Smart Room Controller modeled at the Electronic System Level (ESL), simulating how a processor interacts with a room control unit through a memory-mapped bus interface.

1.2 Approach

- Use SystemC for system-level behavioral modeling
- Use TLM-2.0 (Transaction-Level Modeling) for module communication
- Implement rule-based control logic for fan and lighting
- Follow an APB-like memory-mapped communication style
- Validate system outputs through a processor-driven testbench

2. Project Overview

The Smart Room Controller models a complete smart environment control system at the Electronic System Level. The system reads three environmental inputs — temperature, humidity, and occupancy — from a simulated processor and responds with appropriate control outputs for a fan and lighting unit.

The communication model is based on memory-mapped transactions using the TLM generic payload, following an APB-like abstract protocol. This approach decouples the processor logic from the controller's implementation, mirroring SoC-style design practices.

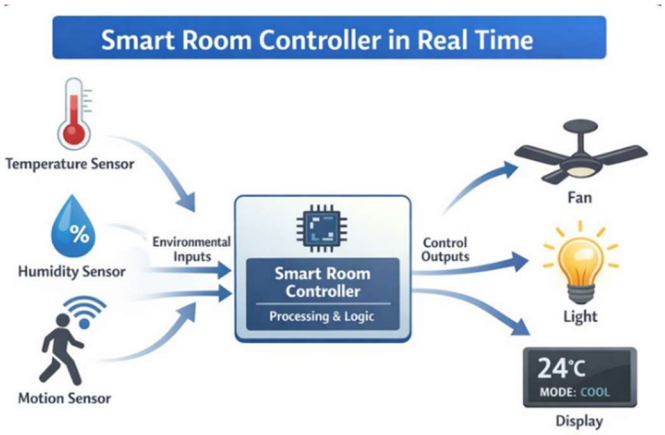

Fig 1: Smart Room Controller in Real Time

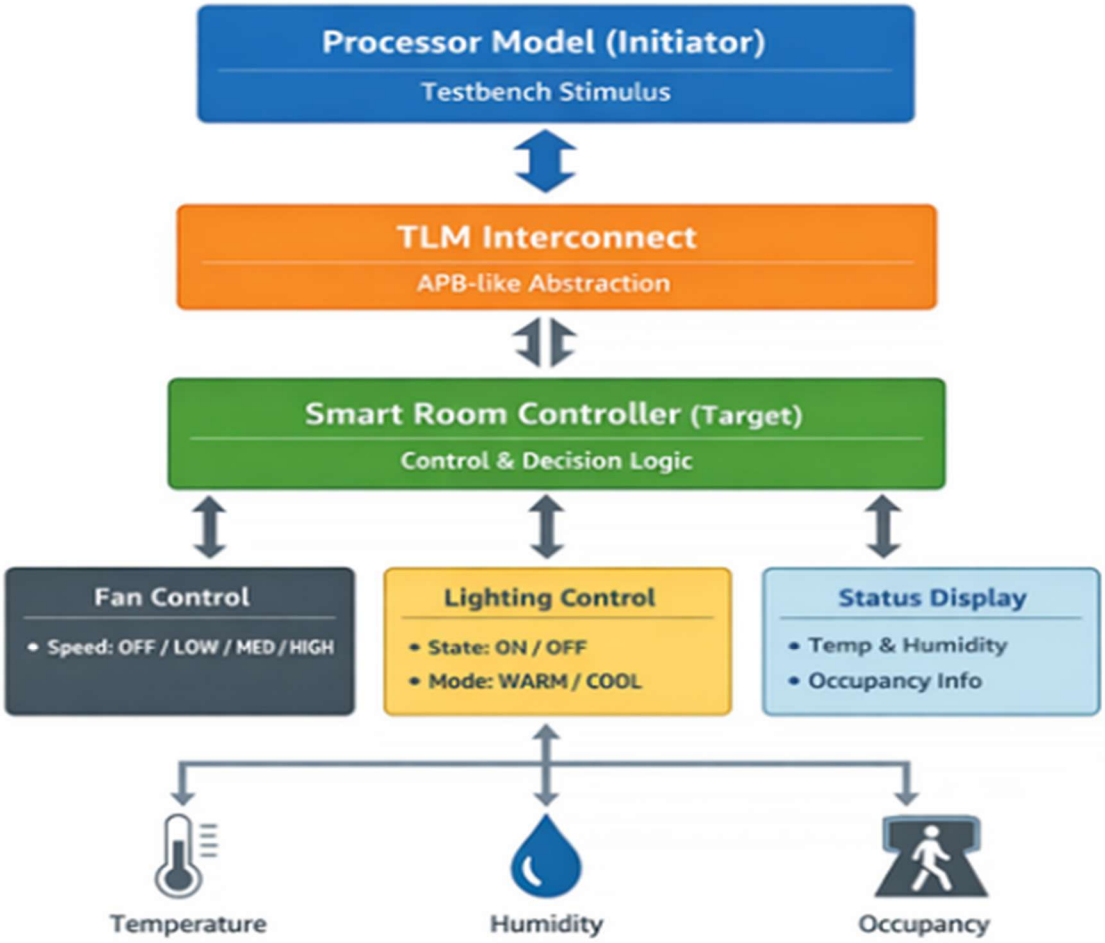

Fig 2: Smart Room Controller: Architecture

2.1 System Inputs

Variable	Type	Size	Description
temp	int	4 bytes	Room temperature in degrees Celsius
hum	int	4 bytes	Relative humidity in percentage (%)
motion	int	4 bytes	Occupancy status (1 = Present, 0 = Absent)

2.2 System Outputs

Variable	Type	Size	Description
fan_speed	int	4 bytes	Fan state: 0=OFF, 1=LOW, 2=MEDIUM, 3=HIGH
light_state	int	4 bytes	Light ON/OFF (1 or 0)
light_color	int	4 bytes	Light mode: 1=Warm, 0=Cool

3. Memory Map (APB-Style)

The system uses memory-mapped registers for all input/output communication. Each register is 4 bytes wide and accessed via the TLM generic payload. The address space is laid out as follows:

Address	Register	Direction	Description
0x00	TEMP_ADDR	Write	Temperature sensor input
0x04	HUM_ADDR	Write	Humidity sensor input
0x08	MOTION_ADDR	Write	Occupancy / motion detection input
0x0C	FAN_ADDR	Read	Fan speed output
0x10	LIGHT_ADDR	Read	Light state output (ON/OFF)
0x14	COLOR_ADDR	Read	Light mode output (Warm/Cool)

Write transactions are used to deliver sensor data to the controller. Read transactions retrieve the controller's computed outputs. Address decoding is handled by the TLM Interconnect module before forwarding to the SmartRoom target.

4. System Architecture

The design follows a three-tier layered SoC-style architecture. The Processor Model acts as the initiator, the TLM Interconnect acts as the bus fabric, and the Smart Room Controller acts as the target peripheral. All inter-module communication uses TLM blocking transport (b_transport).

4.1 Processor Model (Initiator)

The Processor module simulates a CPU at the system level. It runs an SC_THREAD that generates random sensor inputs for each test cycle, writes them to the appropriate memory-mapped addresses, waits for processing, and reads back the system outputs. It communicates using a simple_initiator_socket.

Key characteristics:

- Generates 5 randomized test cycles using rand()
- write(addr, value): Creates a TLM_WRITE_COMMAND transaction and calls b_transport
- read(addr): Creates a TLM_READ_COMMAND transaction, passes a pointer for the return value
- Reports processor observations: Fan | Light | Mode after each cycle

4.2 TLM Interconnect

The Interconnect module acts as the address-decoding bus bridge between the Processor and the SmartRoom controller. It exposes one target socket (cpu_socket) toward the processor and one initiator socket (room_socket) toward the controller.

It performs a simple range check: addresses in the range 0x00 to 0x14 are forwarded to the SmartRoom target. Any address outside this range returns a TLM_ADDRESS_ERROR_RESPONSE. This models a real APB-style peripheral bus decoder.

4.3 Smart Room Controller (Target)

The SmartRoom module is the TLM target (slave). It receives transactions via a simple_target_socket and processes them in the b_transport() callback. It maintains internal state for all sensor inputs and outputs, and runs process_logic() whenever the final input (motion at 0x08) is received.

Internal variables hold the complete system state: temp, hum, motion for inputs, and fan_speed, light_state, light_color for outputs. Each transaction adds a 10 ns delay to model bus latency.

5. Decision Logic (process_logic)

The core rule-based decision engine is triggered once all three sensor inputs have been written. The logic runs sequentially across fan speed, light state, and light mode:

5.1 Fan Control Priority

Motion	Temp > 28°C	Hum > 70%	Fan Output	Reason
0 (Absent)	X	X	OFF	No occupancy — energy saving
1 (Present)	Yes	Yes	HIGH	Temperature has higher priority
1 (Present)	Yes	No	HIGH	Temperature condition satisfied
1 (Present)	No	Yes	MEDIUM	Humidity condition used
1 (Present)	No	No	LOW	Default comfortable case

5.2 Lighting Control

- Occupancy present (motion == 1): Light ON
- No occupancy (motion == 0): Light OFF

5.3 Light Mode

- Temperature > 28°C: Cool mode (light_color = 0)
- Temperature <= 28°C: Warm mode (light_color = 1)

6. Code Structure and Explanation

6.1 Header File (desi.h)

The header file declares the SmartRoom module interface, keeping declaration separate from implementation for clean SoC-style modeling.

```
#ifndef DESI_H
#define DESI_H

#include <systemc>
#include <tlm>
#include <tlm_utils/simple_target_socket.h>

using namespace sc_core;
using namespace tlm;
```

```
struct SmartRoom : sc_module {

    tlm_utils::simple_target_socket<SmartRoom> socket;

    int temp, hum, motion;
    int fan_speed;
    int light_state;
    int light_color;

    SC_CTOR(SmartRoom);

    void b_transport(tlm_generic_payload& trans, sc_time& delay);
    void process_logic();
};

#endif
```

The module inherits from sc_module and uses tlm_utils::simple_target_socket for receiving transactions. Internal state variables are plain integers for simplicity.

6.2 Design File (design.cpp)

The constructor registers b_transport as the socket callback and initializes all internal state to zero. The b_transport function decodes the address and command type:

- TLM_WRITE_COMMAND to 0x00: stores temperature
- TLM_WRITE_COMMAND to 0x04: stores humidity
- TLM_WRITE_COMMAND to 0x08: stores motion and triggers process_logic()
- TLM_READ_COMMAND to 0x0C/0x10/0x14: returns fan/light/mode values

```
#include "desi.h"
#include <iostream>

using namespace std;

#define TEMP_ADDR     0x00
#define HUM_ADDR      0x04
#define MOTION_ADDR   0x08
#define FAN_ADDR      0x0C
#define LIGHT_ADDR    0x10
#define COLOR_ADDR    0x14

SmartRoom::SmartRoom(sc_module_name name)
    : sc_module(name), socket("socket"),
      temp(0), hum(0), motion(0),
      fan_speed(0), light_state(0), light_color(0)
{
    socket.register_b_transport(this, &SmartRoom::b_transport);
}

void SmartRoom::process_logic() {

    if (motion == 0) fan_speed = 0;
    else if (temp > 28) fan_speed = 3;
    else if (hum > 70) fan_speed = 2;
    else fan_speed = 1;
```

```
    light_state = (motion == 1) ? 1 : 0;
    light_color = (temp > 28) ? 0 : 1;

    //  CLEAN OUTPUT FORMAT
    cout << "\n=========== SMART ROOM CYCLE ===========\n";

    cout << "INPUT:\n";
    cout << " Temp      : " << temp << " °C\n";
    cout << " Humidity  : " << hum  << " %\n";
    cout << " Occupancy : " << (motion ? "Present" : "Absent") << "\n";

    cout << "\nPROCESS:\n";
    cout << " Rule Engine Activated\n";

    cout << "\nOUTPUT:\n";

    cout << " Fan       : ";
    if (fan_speed == 0) cout << "OFF\n";
    else if (fan_speed == 1) cout << "LOW\n";
    else if (fan_speed == 2) cout << "MEDIUM\n";
    else cout << "HIGH\n";

    cout << " Light     : " << (light_state ? "ON" : "OFF") << "\n";
    cout << " Mode      : " << (light_color ? "Warm" : "Cool") << "\n";

    cout << "=========================================\n";
}

void SmartRoom::b_transport(tlm_generic_payload& trans, sc_time& delay)
{
    int* data = reinterpret_cast<int*>(trans.get_data_ptr());
    sc_dt::uint64 addr = trans.get_address();

    if (trans.get_command() == TLM_WRITE_COMMAND) {

        if (addr == TEMP_ADDR) temp = data[0];
        else if (addr == HUM_ADDR) hum = data[0];
        else if (addr == MOTION_ADDR) {
            motion = data[0];
            process_logic();
        }
    }

    else if (trans.get_command() == TLM_READ_COMMAND) {

        if (addr == FAN_ADDR) data[0] = fan_speed;
        else if (addr == LIGHT_ADDR) data[0] = light_state;
        else if (addr == COLOR_ADDR) data[0] = light_color;
    }

    delay += sc_time(10, SC_NS);
    trans.set_response_status(TLM_OK_RESPONSE);
}
```

Data is accessed via reinterpret_cast<int*> on the TLM generic payload's data pointer, which correctly maps the 4-byte int values across the generic payload's unsigned char array.

6.3 Testbench (testbench.cpp)

The testbench instantiates and connects all three modules in sc_main:

```
#include <systemc>
#include <tlm>
#include <tlm_utils/simple_initiator_socket.h>
#include <tlm_utils/simple_target_socket.h>
#include <cstdlib>
#include <ctime>
#include <iostream>

#include "desi.h"

using namespace sc_core;
using namespace tlm;
using namespace std;

// ================= INTERCONNECT =================
struct Interconnect : sc_module {

   tlm_utils::simple_target_socket<Interconnect> cpu_socket;
   tlm_utils::simple_initiator_socket<Interconnect> room_socket;

   SC_CTOR(Interconnect)
      : cpu_socket("cpu_socket"), room_socket("room_socket")
   {
      cpu_socket.register_b_transport(this, &Interconnect::b_transport);
   }

   void b_transport(tlm_generic_payload& trans, sc_time& delay)
   {
      sc_dt::uint64 addr = trans.get_address();

      // Simple address decode (only one peripheral here)
      if (addr <= 0x14) {
         room_socket->b_transport(trans, delay);
      }
      else {
         trans.set_response_status(TLM_ADDRESS_ERROR_RESPONSE);
      }
   }
};

// ================= PROCESSOR =================
#define TEMP_ADDR     0x00
#define HUM_ADDR      0x04
#define MOTION_ADDR   0x08
#define FAN_ADDR      0x0C
#define LIGHT_ADDR    0x10
#define COLOR_ADDR    0x14

struct Processor : sc_module {
```

```
tlm_utils::simple_initiator_socket<Processor> socket;

SC_CTOR(Processor) : socket("socket") {
    SC_THREAD(run);
    srand(time(0));
}

void write(int addr, int value) {

    tlm_generic_payload trans;
    sc_time delay = SC_ZERO_TIME;

    trans.set_command(TLM_WRITE_COMMAND);
    trans.set_address(addr);
    trans.set_data_ptr(reinterpret_cast<unsigned char*>(&value));
    trans.set_data_length(sizeof(int));

    socket->b_transport(trans, delay);
}

int read(int addr) {

    int value = 0;

    tlm_generic_payload trans;
    sc_time delay = SC_ZERO_TIME;

    trans.set_command(TLM_READ_COMMAND);
    trans.set_address(addr);
    trans.set_data_ptr(reinterpret_cast<unsigned char*>(&value));
    trans.set_data_length(sizeof(int));

    socket->b_transport(trans, delay);
    return value;
}

void run() {

    for (int i = 0; i < 5; i++) {

        wait(20, SC_NS);

        int temp = 20 + rand() % 20;
        int hum  = 40 + rand() % 50;
        int motion = rand() % 2;

        write(TEMP_ADDR, temp);
        write(HUM_ADDR, hum);
        write(MOTION_ADDR, motion);

        wait(5, SC_NS);

        int fan   = read(FAN_ADDR);
        int light = read(LIGHT_ADDR);
        int color = read(COLOR_ADDR);

        cout << "\nProcessor Observation:\n";
```

```
        cout << " Fan   : " << fan
            << " | Light: " << light
            << " | Mode: " << (color ? "Warm" : "Cool") << "\n";
      }
   }
};

// ================= MAIN =================
int sc_main(int argc, char* argv[])
{
   SmartRoom room("SmartRoom");
   Interconnect bus("APB_Interconnect");
   Processor cpu("Processor");

   cpu.socket.bind(bus.cpu_socket);
   bus.room_socket.bind(room.socket);

   sc_start();
   return 0;
}
```

The Processor's run() SC_THREAD iterates 5 cycles, waiting 20 ns between each. Random inputs are generated: temperature 20–39°C, humidity 40–89%, and motion 0 or 1. After sending all three inputs, it waits 5 ns and reads back the outputs.

7. Environment Setup

Component	Detail
Simulation Platform	EDA Playground (edaplayground.com)
Language	C++ (Standard: C++14)
SystemC Version	SystemC 2.3.3-Accellera
TLM Version	TLM-2.0
Compiler Flags	-std=c++14 -lsystemc
Compile Target	g++ -o sim -std=c++14 -lsystemc *.cpp

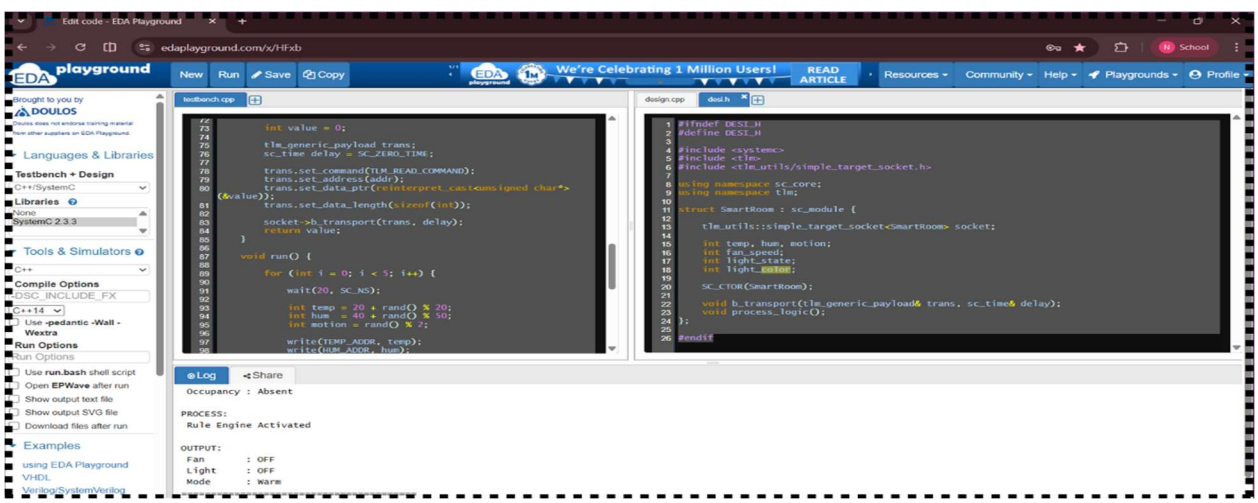

Fig 3: EDA Playground

8. Simulation Results

The simulation was executed on EDA Playground using SystemC 2.3.3, TLM-2.0, and C++14. The following sample outputs demonstrate the controller responding correctly to varied environmental conditions:

```
[2026-02-24 17:05:53 UTC] g++ -o sim -lsystemc *.cpp  && echo "Co
Compile done. Starting run...

        SystemC 2.3.3-Accellera --- May 23 2020 14:33:56
        Copyright (c) 1996-2018 by all Contributors,
        ALL RIGHTS RESERVED

--- New Sensor Reading ---
Temperature: 22 °C
Humidity   : 41 %
Motion     : 0
No motion detected ? Fan OFF
Fan Current State = 0

--- New Sensor Reading ---
Temperature: 37 °C
Humidity   : 65 %
Motion     : 0
No motion detected ? Fan OFF
Fan Current State = 0

--- New Sensor Reading ---
Temperature: 34 °C
Humidity   : 71 %
Motion     : 0
No motion detected ? Fan OFF
Fan Current State = 0

--- New Sensor Reading ---
Temperature: 29 °C
Humidity   : 62 %
Motion     : 1
Comfortable ? Low Speed
Fan Current State = 1

--- New Sensor Reading ---
Temperature: 25 °C
Humidity   : 58 %
Motion     : 1
Comfortable ? Low Speed
Fan Current State = 1

Simulation Complete
Done
```

Cycle 1 — No Occupancy

Parameter	Value
Temperature	33°C
Humidity	50%
Occupancy	Absent
Fan Output	OFF
Light	OFF
Mode	Cool (Temp > 28°C)

Cycle 2 — Occupied, High Temperature

Parameter	Value
Temperature	30°C
Humidity	54%
Occupancy	Present
Fan Output	HIGH (Temp > 28°C)
Light	ON
Mode	Cool (Temp > 28°C)

Both outputs confirm the rule engine is operating correctly. When occupancy is absent, all actuators are disabled regardless of temperature or humidity. When occupied with high temperature, the fan runs at HIGH and light mode switches to Cool.

9. Conclusion

This project successfully demonstrates the design and simulation of a Smart Room Controller at the Electronic System Level using SystemC TLM-2.0. The three-module architecture — Processor, Interconnect, and SmartRoom Controller — closely mirrors real SoC design practice, with memory-mapped communication and APB-like bus abstraction.

The rule-based decision engine correctly prioritizes control actions: no power is consumed when the room is unoccupied; temperature takes priority over humidity for fan control; and light mode adapts to thermal comfort. All five test cycles produced outputs consistent with the specified decision table.

The ESL abstraction proved effective in validating system behavior before committing to RTL implementation. Future work could extend this model with additional peripherals (e.g., a thermostat setpoint register), an interrupt mechanism, or waveform-level VCD output for timing analysis.

10. References

1. IEEE Std 1666-2011: SystemC Language Reference Manual
2. OSCI TLM-2.0 Language Reference Manual, Open SystemC Initiative
3. EDA Playground Simulation Environment — edaplayground.com
4. Source Code Repository: https://github.com/Neethu-Jaisan/SmartRoom-Controller-System-C-TLM-Modeling